

Codity — Distributed Job Scheduling Platform

Take-home assignment submission · Node.js + TypeScript + PostgreSQL
Submitted by **Siddarth** · rsiddarth69@gmail.com

GitHub Repository https://github.com/rsiddarth10/codity_ai
Deployed Application _____

Overview

Codity is a production-inspired distributed job scheduler where **PostgreSQL is both the system of record and the queue**. Jobs are claimed atomically by a pool of horizontally-scalable workers using `SELECT ... FOR UPDATE SKIP LOCKED`, retried with configurable backoff, dead-lettered on exhaustion, scheduled (delayed / cron / batch), gated by workflow dependencies, and fully observable (per-attempt history, structured logs, lifecycle audit). It ships as a one-command Docker Compose stack: API + worker pool + scheduler + Postgres + a React dashboard.

- TypeScript / Node 20
- PostgreSQL 16
- pg + node-pg-migrate (raw SQL)
- Express + zod + pino
- JWT + bcryptjs
- cron-parser
- React + Vite + Recharts
- Vitest
- Docker Compose

Deliverables

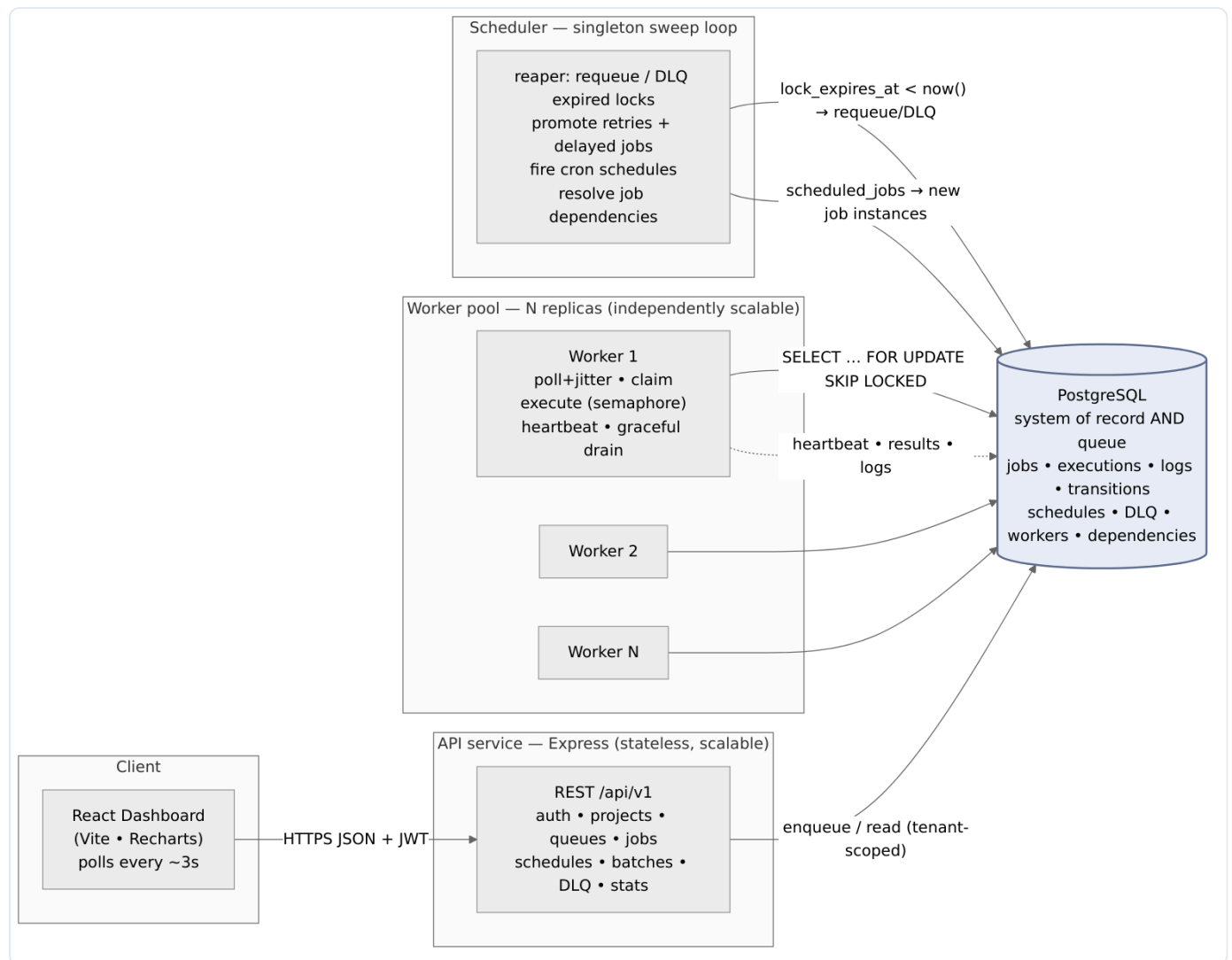
Deliverable	Where it lives
Source code + setup instructions	GitHub repo · README.md · setup section below
Architecture diagram	Below · deliverables/architecture.svg & .png
ER diagram	Below · deliverables/er-diagram.svg & .png
API documentation	OpenAPI 3.0 (27 paths) · live Swagger UI at GET /docs · deliverables/openapi.json
Design decisions document (trade-offs)	DESIGN.md (~4,900 words, written incrementally)
Automated tests for critical functionality	test/ — 63 tests, 15 files, real dockerized Postgres

Evaluation criteria — how each is addressed

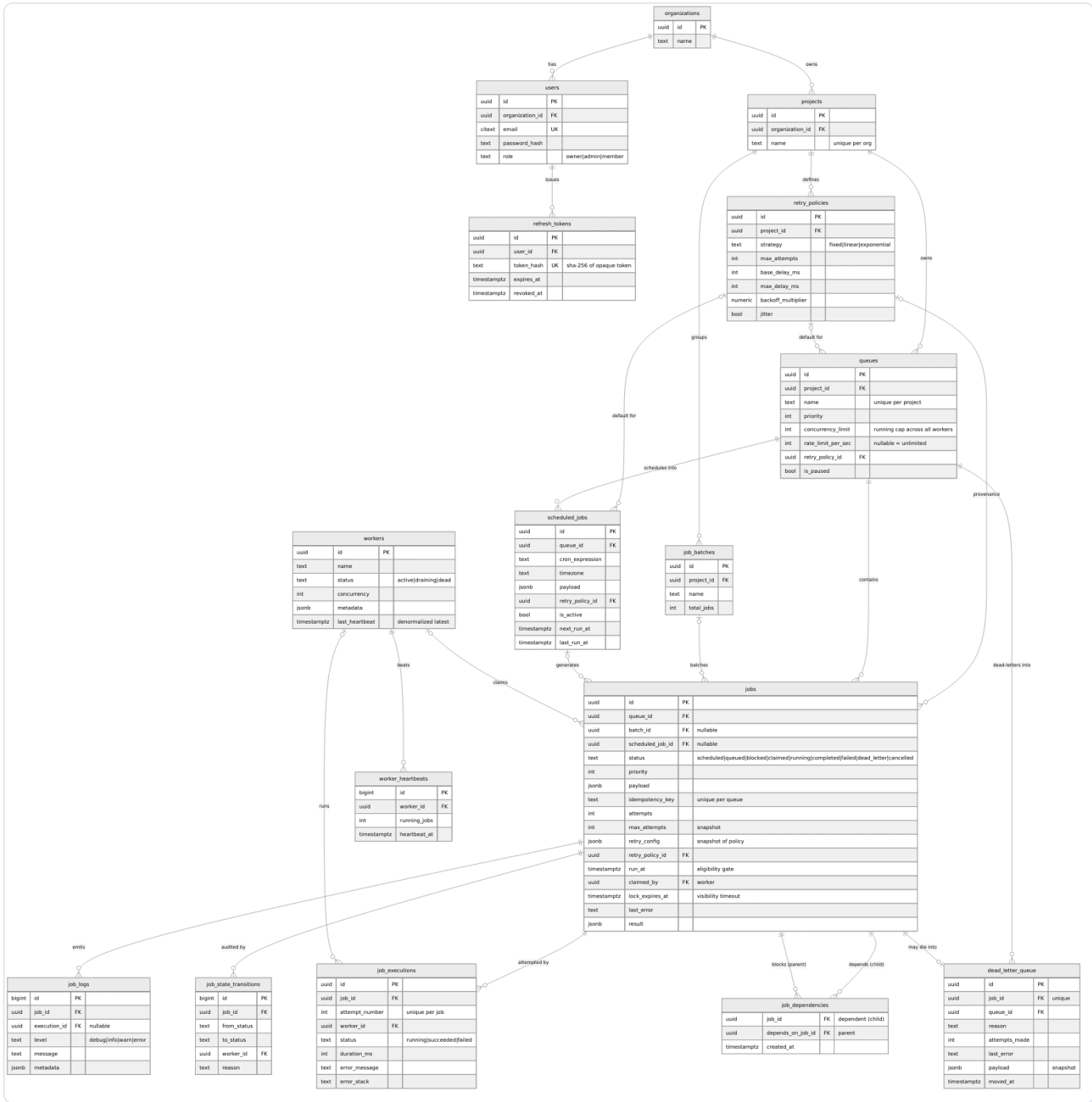
Criteria	Marks	Coverage
----------	-------	----------

System Architecture	20	Monorepo of independently-scalable services (API / worker pool / scheduler / frontend) over a shared @codity/* core; Postgres as single source of truth + queue; clean layering (db = schema, core = data-access, services = thin endpoints). See diagram.
Database Design	20	Normalized 16-table schema, 10 reversible migrations; partial claim index ; retry policy snapshotted onto each job; documented cascade / SET NULL behavior. See ER diagram + schema code.
Backend Engineering	20	Raw, reviewable SQL data-access layer; Express REST API with zod validation, offset pagination, centralized structured errors, pino request logging; JWT auth with rotating hashed refresh tokens; RBAC.
Reliability & Concurrency	15	FOR UPDATE SKIP LOCKED atomic claiming (proven : 500 jobs / 24 claimers → zero duplicates); per-queue advisory lock for exact concurrency + rate limits; heartbeats + reaper for crash recovery; graceful shutdown draining; retries → DLQ; all correctness paths in DB transactions.
Frontend & UX	10	React dashboard: queue health, job explorer (filter + paginate), job detail (timeline / attempts / logs), worker fleet, Dead Letter Queue with one-click retry, queue config, throughput charts; live polling.
API Design	5	RESTful resource URLs + correct verbs/status codes; zod validation; consistent pagination + error envelope; OpenAPI 3.0 served at /docs.
Documentation	5	DESIGN.md (incremental decisions + trade-offs), architecture + ER diagrams, OpenAPI spec, README, this document.
Testing	5	63 automated tests on real dockerized Postgres: concurrency proof, crash recovery, retry/DLQ, scheduling, dependencies, RBAC, rate limiting, full API.

Architecture diagram



ER diagram (full schema)



Design decisions & major trade-offs

- **Postgres-as-queue, not a broker** — one transactional source of truth for state + history + queue; SKIP LOCKED gives correct multi-consumer claiming. Trade-off: lower ceiling than a dedicated broker at extreme rates (mitigation: queue sharding).
- **Atomic claiming + exact limits** — FOR UPDATE SKIP LOCKED guarantees no double-claim; a per-queue transaction-scoped advisory lock makes the in-flight count exact so concurrency/rate limits can't be exceeded. Cost: claims for one queue serialize (sub-ms), execution stays fully concurrent.
- **Retry policy snapshotted onto the job at enqueue** — retries are deterministic; editing a queue's policy never changes in-flight jobs.
- **Enum-like columns are TEXT + CHECK** (not native ENUM) — adding a value is a cheap, reversible migration.
- **Crash recovery via lock expiry** — heartbeats extend a per-job lock; the reaper requeues (or dead-letters) expired jobs. One mechanism covers hard crashes and hangs.
- **Cron advances from now()** — a scheduler that was down fires once and moves on instead of backfilling a storm of missed runs.
- **Workflow deps use a blocked status** — blocked jobs are invisible to the claim query, so the hot path + partial index stay untouched; cycles are impossible by construction.
- **Hard-delete + CASCADE** for simplicity, with soft-delete noted as the production retention alternative.

Full rationale for every decision is in **DESIGN.md** (\$0–\$8).

Testing (what is proven, not asserted)

- **Atomic claiming under real concurrency** — 500 jobs / 24 concurrent claimers → each claimed exactly once.
- **SKIP LOCKED semantics** — a locked row is skipped, not blocked.
- **Concurrency + rate limits** — never exceeded under load; pause/resume respected.
- **Crash recovery** — a "dead" worker's job is requeued (or dead-lettered if exhausted); healthy heartbeats are left alone.
- **Graceful shutdown** — in-flight jobs drain; nothing stuck in claimed/running.
- **Retries → DLQ → manual revive**; backoff math (fixed/linear/exponential + cap + jitter).
- **Scheduling** — cron next-run, delayed promotion, cron firing without double-fire, batch rollup.
- **Workflow dependencies, RBAC, rate limiting, and the full REST API** (auth, tenant isolation, validation, pagination, idempotency).

Run with `npm test` (spins up an isolated `codity_test` database automatically).

Source code with setup

Setup — the full stack runs with one command:

```
git clone https://github.com/rsiddarth10/codity_ai.git && cd codity_ai
npm install
cp .env.example .env

# Whole stack (Postgres + API + worker + scheduler + dashboard):
docker compose up -d --build
npm run migrate:up          # apply the schema

#   API ..... http://localhost:4000      (Swagger UI at /docs)
#   Dashboard .... http://localhost:5173
#   Scale workers: docker compose up -d --scale worker=3

# Run the test suite (needs Postgres up):
npm test                    # 63 tests, real Postgres
```

The most important source is embedded below (schema + claim query + retry backoff + worker engine). The complete, runnable codebase — API, scheduler, React dashboard, all 15 test files — is on [GitHub](#).

Database schema — the jobs table & the partial CLAIM index (migration 0005)

```
/* eslint-disable camelcase */

/**
 * Phase 1 – migration 5/8: job batches and the core jobs table.
 *
 * This is the heart of the system. Key modeling decisions:
 *
 *   Status state machine (TEXT + CHECK):
 *   - queued -> claimed -> running -> completed
 *   - running -> failed -> queued (retry, run_at pushed into the future)
 *   - failed -> dead_letter (max attempts exhausted)
 *   - scheduled -> queued (a delayed/one-shot future job becomes eligible)
 *   - any -> cancelled (manual)
 *
 * Eligibility gating: the claim query selects status='queued' AND run_at <= now().
 *   - Immediate jobs: status='queued', run_at=now().
 *   - Delayed / one-shot scheduled jobs: status='scheduled', run_at in the future;
 *     the scheduler promotes them to 'queued' when due (explicit state, clean claim).
 *
 * Retry snapshot: max_attempts + retry_config are SNAPSHOTTED onto the job at enqueue
 *   from the effective policy (job override > queue policy > system default). Editing a
 *   policy afterwards never changes in-flight jobs – retries are deterministic.
 *
 * Lock / visibility timeout: lock_expires_at is set when a job is claimed and extended
 *   by worker heartbeats. The reaper requeues any claimed/running job whose lock has
 *   expired – this recovers jobs from hard-crashed workers without needing to first
 *   detect the worker is dead.
 *
 * Idempotency: (queue_id, idempotency_key) is unique among non-null keys, so a repeated
 *   submission with the same key returns the existing job instead of enqueueing a
 *   duplicate. "Idempotent" here = at-most-once *enqueue* per key; see DESIGN.md.
 */

exports.shorthands = undefined;

const JOB_STATUSES = [
  'scheduled',
  'queued',
  'claimed',
  'running',
  'completed',
  'failed',
  'dead_letter',
  'cancelled',
]
```

```

];

exports.up = (pgm) => {
  pgm.createTable('job_batches', {
    id: { type: 'uuid', primaryKey: true, default: pgm.func('gen_random_uuid()') },
    project_id: {
      type: 'uuid',
      notNull: true,
      references: 'projects(id)',
      onDelete: 'CASCADE',
    },
    name: { type: 'text', notNull: true },
    total_jobs: { type: 'integer', notNull: true, default: 0, check: 'total_jobs >= 0' },
    created_at: { type: 'timestampz', notNull: true, default: pgm.func('now()') },
  });
  pgm.createIndex('job_batches', 'project_id');

  pgm.createTable('jobs', {
    id: { type: 'uuid', primaryKey: true, default: pgm.func('gen_random_uuid()') },
    queue_id: {
      type: 'uuid',
      notNull: true,
      references: 'queues(id)',
      onDelete: 'CASCADE',
    },
    // Batch membership is optional; if a batch is deleted, jobs survive (SET NULL).
    batch_id: {
      type: 'uuid',
      references: 'job_batches(id)',
      onDelete: 'SET NULL',
    },
    // FK to scheduled_jobs is added in migration 0007 (table created there).
    scheduled_job_id: { type: 'uuid' },
    status: {
      type: 'text',
      notNull: true,
      default: 'queued',
      check: `status IN (${JOB_STATUSES.map((s) => ` '${s}' `).join(', ')} )`,
    },
    priority: { type: 'integer', notNull: true, default: 0 },
    payload: { type: 'jsonb', notNull: true, default: '{}' },
    idempotency_key: { type: 'text' },
    attempts: { type: 'integer', notNull: true, default: 0, check: 'attempts >= 0' },
    max_attempts: { type: 'integer', notNull: true, default: 3, check: 'max_attempts >= 1' },
    // Snapshotted effective retry policy: { strategy, base_delay_ms, max_delay_ms, multiplier, jitter }.
    retry_config: {
      type: 'jsonb',
      notNull: true,
      default: pgm.func(
        `('${"strategy": "exponential", "base_delay_ms": 1000, "max_delay_ms": 60000, "multiplier": 2, "jitter": true}':::jsonb`,
      ),
    },
    // Provenance only; effective values live in retry_config/max_attempts.
    retry_policy_id: {
      type: 'uuid',
      references: 'retry_policies(id)',
      onDelete: 'SET NULL',
    },
    run_at: { type: 'timestampz', notNull: true, default: pgm.func('now()') },
    claimed_by: {
      type: 'uuid',
      references: 'workers(id)',
      onDelete: 'SET NULL',
    },
    claimed_at: { type: 'timestampz' },
    started_at: { type: 'timestampz' },
    completed_at: { type: 'timestampz' },
    lock_expires_at: { type: 'timestampz' },
    last_error: { type: 'text' },
    result: { type: 'jsonb' },
    created_at: { type: 'timestampz', notNull: true, default: pgm.func('now()') },
    updated_at: { type: 'timestampz', notNull: true, default: pgm.func('now()') },
  });

  // --- Indexes on the hot paths ---

```

```

// 1) THE CLAIM INDEX. Matches the claim query exactly:
//   WHERE queue_id=$1 AND status='queued' AND run_at<=now()
//   ORDER BY priority DESC, run_at ASC, created_at ASC
//   Partial on status='queued' keeps it small and cache-hot.
pgm.createIndex(
  'jobs',
  [
    { name: 'queue_id' },
    { name: 'priority', sort: 'DESC' },
    { name: 'run_at', sort: 'ASC' },
    { name: 'created_at', sort: 'ASC' },
  ],
  { name: 'idx_jobs_claim', where: "status = 'queued'" },
);

// 2) REAPER INDEX: find claimed/running jobs whose lock has expired.
pgm.createIndex('jobs', 'lock_expires_at', {
  name: 'idx_jobs_reaper',
  where: "status IN ('claimed', 'running'",
});

// 3) SCHEDULER PROMOTION: find scheduled jobs that are now due.
pgm.createIndex('jobs', 'run_at', {
  name: 'idx_jobs_scheduled_due',
  where: "status = 'scheduled'",
});

// 4) IDEMPOTENCY: at most one live job per (queue, key).
pgm.createIndex('jobs', ['queue_id', 'idempotency_key'], {
  name: 'idx_jobs_idempotency',
  unique: true,
  where: 'idempotency_key IS NOT NULL',
});

// 5) DASHBOARD list/filter/paginate by queue + status, newest first.
pgm.createIndex('jobs', [{ name: 'queue_id' }, { name: 'status' }, { name: 'created_at', sort: 'DESC' }], {
  name: 'idx_jobs_queue_status_created',
});

// 6) BATCH rollups.
pgm.createIndex('jobs', 'batch_id', {
  name: 'idx_jobs_batch',
  where: 'batch_id IS NOT NULL',
});

pgm.sql(`
  CREATE TRIGGER trg_jobs_updated_at BEFORE UPDATE ON jobs
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
`);
};

exports.down = (pgm) => {
  pgm.dropTable('jobs');
  pgm.dropTable('job_batches');
};

```

THE claim query — atomic, no-double-claim, concurrency-safe (packages/core/src/claim.ts)

```

import type { Pool } from '@codity/db';
import { withTransaction, logTransition } from './tx.js';
import type { JobRow } from './types.js';

/**
 * _____
 * THE CLAIM QUERY — the heart of the scheduler.
 * _____
 *
 * Two safety properties, two mechanisms:
 *
 * (1) NO TWO WORKERS EVER CLAIM THE SAME JOB.
 *   Guaranteed by `FOR UPDATE SKIP LOCKED` on the `claimable` CTE: each concurrent
 *   claimer row-locks the candidate jobs it selects and SKIPS any row already locked
 *   by another in-flight claim. Locked rows are invisible, not blocking — so N

```



```

*   claimers partition the queue into disjoint sets with zero coordination and zero
*   double-delivery. This holds with or without mechanism (2).
*
* (2) THE QUEUE CONCURRENCY LIMIT IS NEVER EXCEEDED.
*   "max running across ALL workers" requires an EXACT count of in-flight
*   (claimed+running) jobs at claim time. A bare count is a read that races with
*   other claimers' commits. We serialize the claim critical section PER QUEUE with a
*   transaction-scoped advisory lock keyed by the queue id (see claimJobs). Within
*   that lock the in-flight count is exact, so `capacity = limit - inflight` is
*   correct and we can never overcommit. Different queues use different advisory keys
*   and claim fully in parallel; execution is always concurrent. Claims are just fast
*   row updates, so serializing them per queue is cheap. (Trade-off discussed in
*   DESIGN.md §2.)
*
* Ordering: priority DESC (higher first), then oldest run_at, then FIFO by created_at.
* Eligibility: status='queued' AND run_at<=now(); a paused queue yields capacity 0.
* Matches the partial index idx_jobs_claim(queue_id, priority DESC, run_at, created_at)
* WHERE status='queued'.
*/
export const CLAIM_JOBS_SQL = `
WITH cfg AS (
  SELECT concurrency_limit, is_paused, rate_limit_per_sec
  FROM queues
  WHERE id = $1
),
inflight AS (
  SELECT count(*)::int AS n
  FROM jobs
  WHERE queue_id = $1
  AND status IN ('claimed', 'running')
),
-- Jobs claimed within the last rolling second (for rate limiting). Exact because claims
-- for a queue are serialized by the advisory lock in claimJobs().
recent AS (
  SELECT count(*)::int AS n
  FROM jobs
  WHERE queue_id = $1
  AND claimed_at > now() - interval '1 second'
),
capacity AS (
  SELECT CASE
    WHEN NOT EXISTS (SELECT 1 FROM cfg)          THEN 0
    WHEN (SELECT is_paused FROM cfg)              THEN 0
    ELSE LEAST(
      $2::int,
      -- concurrency headroom
      GREATEST(0, (SELECT concurrency_limit FROM cfg) - (SELECT n FROM inflight)),
      -- rate-limit headroom (NULL => unlimited)
      CASE WHEN (SELECT rate_limit_per_sec FROM cfg) IS NULL THEN $2::int
            ELSE GREATEST(0, (SELECT rate_limit_per_sec FROM cfg) - (SELECT n FROM recent))
        END
    )
  END AS lim
),
claimable AS (
  SELECT j.id
  FROM jobs j
  WHERE j.queue_id = $1
    AND j.status = 'queued'
    AND j.run_at <= now()
  ORDER BY j.priority DESC, j.run_at ASC, j.created_at ASC
  FOR UPDATE SKIP LOCKED
  LIMIT (SELECT lim FROM capacity)
)
UPDATE jobs
  SET status = 'claimed',
      claimed_by = $3,
      claimed_at = now(),
      lock_expires_at = now() + ($4::numeric * interval '1 millisecond'),
      updated_at = now()
  FROM claimable
 WHERE jobs.id = claimable.id
RETURNING jobs.*;
`;

```

```

export interface ClaimJobsInput {
  queueId: string;
  workerId: string;
  /** Upper bound on how many jobs to claim this poll (further capped by capacity). */
  batchSize: number;
  /** How long the claim lock is valid before the reaper may reclaim it. */
  lockDurationMs: number;
}

/**
 * Atomically claim up to `batchSize` jobs from a queue for a worker, respecting the
 * queue's concurrency limit and pause state. Returns the claimed job rows (possibly
 * empty). Each claim runs in its own transaction under a per-queue advisory lock.
 */
export async function claimJobs(pool: Pool, input: ClaimJobsInput): Promise<JobRow[]> {
  return withTransaction(pool, async (client) => {
    // Serialize the claim critical section for THIS queue only (mechanism 2 above).
    // hashtextextended(uuid::text) -> stable bigint advisory key.
    await client.query('SELECT pg_advisory_xact_lock(hashtextextended($1::text, 0))', [
      input.queueId,
    ]);

    const { rows } = await client.query<JobRow>(CLAIM_JOBS_SQL, [
      input.queueId,
      input.batchSize,
      input.workerId,
      input.lockDurationMs,
    ]);

    for (const job of rows) {
      await logTransition(client, job.id, 'queued', 'claimed', input.workerId, 'claimed by worker');
    }
    return rows;
  });
}

```

Retry backoff calculator — fixed / linear / exponential + cap + jitter (packages/core/src/retry.ts)

```

import type { RetryConfig } from './types.js';

/**
 * Compute the delay (ms) before the next attempt, given the snapshotted retry policy and
 * the attempt number that just FAILED (1-based). Pure and deterministic (inject `rng` in
 * tests). Strategies:
 *
 *   * fixed:      base
 *   * linear:     base * attempt
 *   * exponential: base * multiplier^(attempt-1)
 *
 * Then the result is capped at `max_delay_ms` (if set), and — when `jitter` is on —
 * "equal jitter" is applied: keep half the delay and randomize the other half, i.e. the
 * returned value lands in [delay/2, delay]. Equal jitter spreads retry storms across
 * workers while still guaranteeing a minimum backoff (unlike full jitter, which can return
 * ~0 and hammer the queue).
 */
export function computeBackoffMs(
  config: RetryConfig,
  attempt: number,
  rng: () => number = Math.random,
): number {
  const n = Math.max(1, Math.floor(attempt));

  let delay: number;
  switch (config.strategy) {
    case 'fixed':
      delay = config.base_delay_ms;
      break;
    case 'linear':
      delay = config.base_delay_ms * n;
      break;
    case 'exponential':
      delay = config.base_delay_ms * Math.pow(config.multiplier, n - 1);
      break;
    default:

```

```

    delay = config.base_delay_ms;
  }

  if (config.max_delay_ms != null) {
    delay = Math.min(delay, config.max_delay_ms);
  }

  if (config.jitter) {
    const half = delay / 2;
    delay = half + rng() * half;
  }

  return Math.max(0, Math.round(delay));
}

```

Worker engine — poll / claim / semaphore / heartbeat / graceful drain (packages/worker/src/engine.ts)

```

import type { Pool } from '@codity/db';
import {
  claimJobs,
  startJob,
  completeJob,
  failJob,
  heartbeatJobs,
  heartbeatWorker,
  registerWorker,
  setWorkerStatus,
  listQueueIdsByPriority,
  addJobLog,
  type JobRow,
} from '@codity/core';
import type { Logger } from '@codity/shared';
import { type HandlerRegistry, type JobContext } from './handlers.js';

export interface WorkerEngineConfig {
  /** Explicit queue ids to poll. If omitted/empty, discover all non-paused queues. */
  queueIds?: string[];
  /** Max jobs this worker runs concurrently (the in-process semaphore size). */
  concurrency: number;
  pollIntervalMs: number;
  pollJitterMs: number;
  heartbeatIntervalMs: number;
  lockDurationMs: number;
  shutdownTimeoutMs: number;
  workerName: string;
}

export type StopOutcome = 'drained' | 'timeout';

/**
 * A single worker process's engine. Responsibilities:
 * - poll assigned queues on an interval WITH JITTER (avoid thundering herd),
 * - claim only up to the free semaphore slots (never overcommit),
 * - execute jobs CONCURRENTLY (bounded by `concurrency`), not one-at-a-time,
 * - heartbeat the worker AND extend in-flight job locks so the reaper leaves them be,
 * - shut down gracefully: stop polling, drain in-flight up to a timeout, then exit.
 *
 * The engine is deliberately transport-free and injectable (pool, handlers, logger) so it
 * can be unit/integration-tested in-process; `index.ts` is the thin env-driven entrypoint.
 */
export class WorkerEngine {
  private workerId = '';
  /** jobId -> in-flight execution promise (its size IS the semaphore counter). */
  private readonly inFlight = new Map<string, Promise<void>>();
  private shuttingDown = false;
  private pollTimer?: ReturnType<typeof setTimeout>;
  private heartbeatTimer?: ReturnType<typeof setInterval>;
  private readonly abortController = new AbortController();

  constructor(
    private readonly pool: Pool,
    private readonly config: WorkerEngineConfig,
    private readonly handlers: HandlerRegistry,
    private readonly logger: Logger,
  ) {}

```

```

) {}

get id(): string {
    return this.workerId;
}

get inFlightCount(): number {
    return this.inFlight.size;
}

async start(): Promise<void> {
    const worker = await registerWorker(this.pool, {
        name: this.config.workerName,
        concurrency: this.config.concurrency,
        metadata: {
            host: process.env.HOSTNAME ?? 'local',
            pid: process.pid,
            queues: this.config.queueIds ?? 'all',
        },
    });
    this.workerId = worker.id;
    this.logger.info({ workerId: this.workerId, concurrency: this.config.concurrency }, 'worker started');

    this.startHeartbeat();
    this.scheduleNextPoll(0); // first poll immediately
}

private availableSlots(): number {
    return this.config.concurrency - this.inFlight.size;
}

/** One poll cycle: fill free slots by claiming across queues (priority order). */
private async pollOnce(): Promise<void> {
    if (this.shuttingDown) return;
    let slots = this.availableSlots();
    if (slots <= 0) return;

    const queueIds =
        this.config.queueIds && this.config.queueIds.length > 0
        ? this.config.queueIds
        : await listQueueIdsByPriority(this.pool);

    for (const queueId of queueIds) {
        if (slots <= 0 || this.shuttingDown) break;
        const claimed = await claimJobs(this.pool, {
            queueId,
            workerId: this.workerId,
            batchSize: slots,
            lockDurationMs: this.config.lockDurationMs,
        });
        for (const job of claimed) {
            slots -= 1;
            this.launch(job);
        }
    }
}

/** Begin executing a claimed job; tracks it in the semaphore until settled. */
private launch(job: JobRow): void {
    const promise = this.execute(job).finally(() => {
        this.inFlight.delete(job.id);
    });
    this.inFlight.set(job.id, promise);
}

private async execute(job: JobRow): Promise<void> {
    let executionId: string | null = null;
    try {
        const started = await startJob(this.pool, job.id, this.workerId);
        if (!started) {
            // Lost the job between claim and start (e.g. reaped). Nothing to do.
            this.logger.warn({ jobId: job.id }, 'could not start job (no longer owned)');
            return;
        }
        executionId = started.executionId;
    }
}

```

```

const ctx: JobContext = {
  job: started.job,
  signal: this.abortController.signal,
  log: (level, message, metadata) =>
    addJobLog(this.pool, { jobId: job.id, executionId, level, message, metadata }),
};

const handler = this.handlers.resolve(started.job);
const result = await handler(ctx);
await completeJob(this.pool, job.id, this.workerId, result ?? null);
this.logger.debug({ jobId: job.id }, 'job completed');
} catch (err) {
  const error = err instanceof Error ? err : new Error(String(err));
  await failJob(this.pool, job.id, this.workerId, error).catch((e) =>
    this.logger.error({ jobId: job.id, err: e }, 'failed to record job failure'),
  );
  if (executionId) {
    await addJobLog(this.pool, {
      jobId: job.id,
      executionId,
      level: 'error',
      message: error.message,
      metadata: { stack: error.stack },
    }).catch(() => {});
  }
  this.logger.warn({ jobId: job.id, err: error.message }, 'job failed');
}
}

private scheduleNextPoll(delayMs?: number): void {
  if (this.shuttingDown) return;
  const wait =
    delayMs !== undefined
      ? delayMs
      : this.config.pollIntervalMs + (Math.random() * 2 - 1) * this.config.pollJitterMs;
  this.pollTimer = setTimeout(() => {
    void this.pollOnce()
      .catch((err) => this.logger.error({ err }, 'poll cycle error'))
      .finally(() => this.scheduleNextPoll());
  }, Math.max(0, wait));
}

private startHeartbeat(): void {
  this.heartbeatTimer = setInterval(() => {
    void this.beat().catch((err) => this.logger.error({ err }, 'heartbeat error'));
  }, this.config.heartbeatIntervalMs);
}

/** Refresh the worker heartbeat and extend locks on all in-flight jobs. */
private async beat(): Promise<void> {
  const jobIds = [...this.inFlight.keys()];
  await heartbeatWorker(this.pool, this.workerId, jobIds.length);
  if (jobIds.length > 0) {
    await heartbeatJobs(this.pool, this.workerId, jobIds, this.config.lockDurationMs);
  }
}

/**
 * Graceful shutdown: stop claiming, let in-flight jobs finish (up to the timeout),
 * then stop heartbeating and mark the worker stopped. Jobs still running past the
 * timeout are abandoned and recovered by the reaper via lock expiry – never lost.
 */
async stop(): Promise<StopOutcome> {
  if (this.shuttingDown) return 'drained';
  this.shuttingDown = true;
  if (this.pollTimer) clearTimeout(this.pollTimer);
  await setWorkerStatus(this.pool, this.workerId, 'draining').catch(() => {});

  const outcome = await this.drain();

  if (this.heartbeatTimer) clearInterval(this.heartbeatTimer);
  await setWorkerStatus(this.pool, this.workerId, 'dead').catch(() => {});
  this.logger.info({ outcome, remaining: this.inFlight.size }, 'worker stopped');
  return outcome;
}

```

```

private async drain(): Promise<StopOutcome> {
  const drained = Promise.allSettled([...this.inFlight.values()]).then<StopOutcome>(() => 'drained');
  let timer: ReturnType<typeof setTimeout>;
  const timedOut = new Promise<StopOutcome>((resolve) => {
    timer = setTimeout(() => resolve('timeout'), this.config.shutdownTimeoutMs);
  });
  const outcome = await Promise.race([drained, timedOut]);
  clearTimeout(timer!);
  if (outcome === 'timeout') {
    // Signal cooperative handlers to abort; leftovers will be reaped via lock expiry.
    this.abortController.abort();
    this.logger.warn({ remaining: this.inFlight.size }, 'drain timed out; leftovers will be reaped');
  }
  return outcome;
}
}

```

Codity · full source, docs, and tests at https://github.com/rsiddarth10/codity_ai